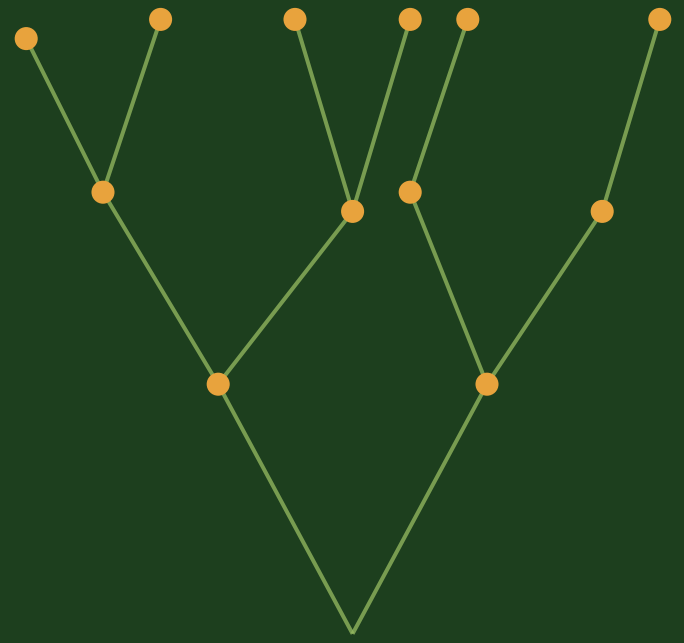
SUPERVISED LEARNING · CLASSIFICATION

Decision Trees

How machines learn to ask the right questions, in the right order

Root Nodes · Entropy · Information Gain · Pruning



What Is a Decision Tree?

1. **A supervised classification model shaped like a tree**

   It predicts an output (target) variable using the input variables in a feature vector.

2. **One of the most widely adopted classification algorithms**

   Prized for fast execution and rules that are easy for humans to interpret.

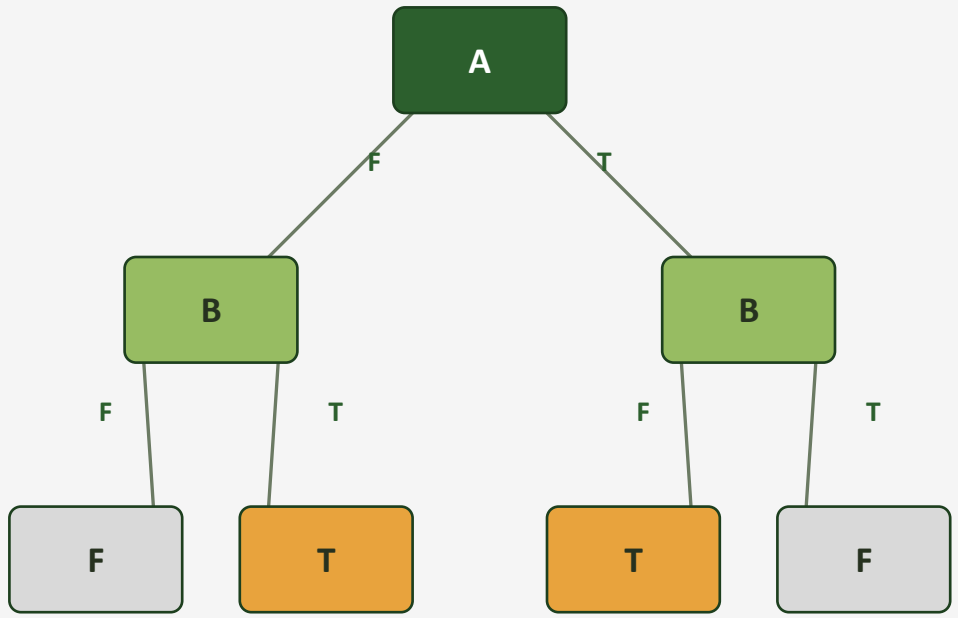
3. **Built for multi-dimensional analysis**

   Handles multiple input features and multiple output classes at once.

4. **Learns by asking a sequence of questions**

   Each internal question narrows down the data until a decision is reached.

GENERIC STRUCTURE



A = root node B = branch node T/F = leaf nodes

Anatomy of a Decision Tree

1

Root Node

The first node of the tree. Represents the entire data set and tests the single strongest predicting feature first.

2

Branch Node

An internal node that tests one attribute. Every possible value (or range of values) of that attribute leads to a child branch.

3

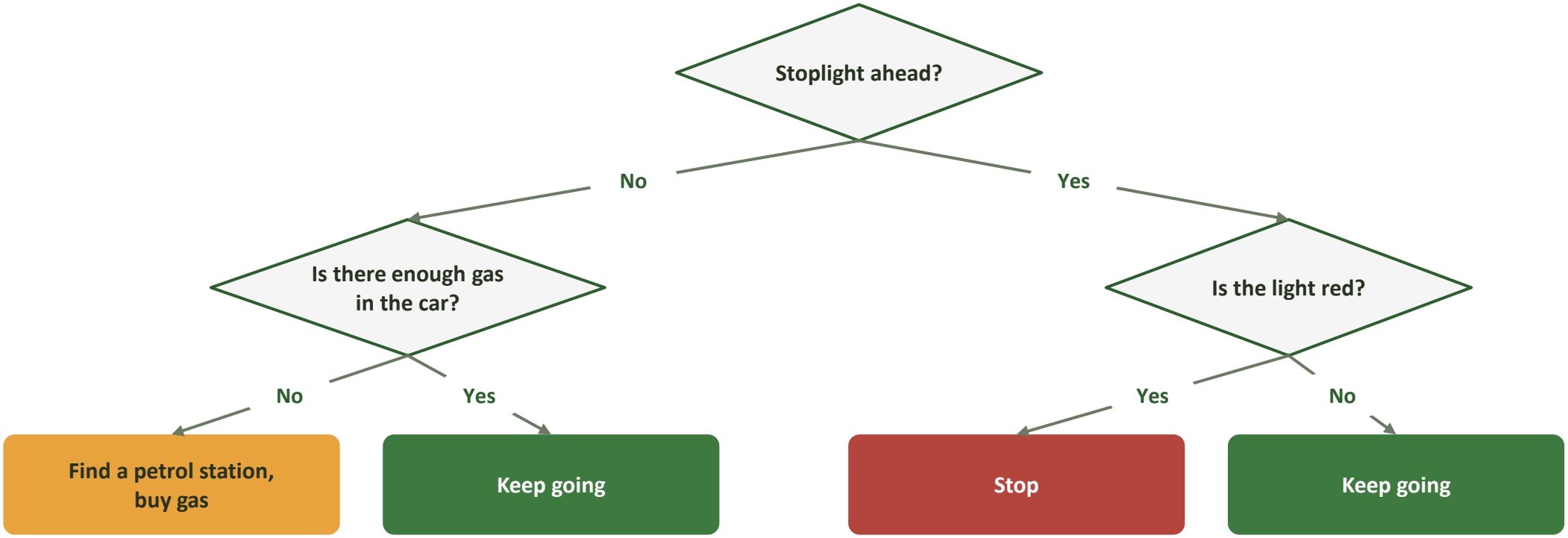
Leaf Node

A terminal node with no further children. It assigns the final classification — the predicted value of the output variable.

How it flows: starting at the root, follow the branch matching each attribute's value, level by level, until a leaf is reached — that leaf's label is the prediction.

WORKED EXAMPLE

Everyday Decision Trees: Should the Car Keep Going?



Reading the tree: each diamond is a decision (branch node); each colored box is an outcome (leaf node). A red signal forces a stop; low fuel routes the car to the nearest gas station.

THE ALGORITHM

Building a Tree: Recursive Partitioning

Decision trees are grown from training data using recursive partitioning — repeatedly splitting the data set into purer and purer subsets.

1. Start at the root node — which is simply the entire training data set.
2. Select the feature that predicts the target class in the strongest way.
3. Split the data set into partitions, one per distinct value of that feature.
4. Repeat the process on each partition, splitting on the next-best feature.
5. Continue until a stopping criterion is reached.

STOPPING CRITERIA

1. All (or most) examples at a node belong to the same class
2. All available features have already been used for splitting
3. The tree has grown to a pre-defined threshold / size limit

CASE STUDY

GTS Campus Recruitment: Will Chandra Get the Offer?

Global Technology Solutions (GTS) shortlisted 18 B.Tech. students for final interviews. **Chandra** wants to predict his own outcome — CGPA: High, Communication: Bad, Aptitude: High, Programming Skills: Bad — using last year's interview results as training data.

CGPA	Communication	Aptitude	Programming Skill	Job Offered?
High	Good	High	Good	Yes
Medium	Good	High	Good	Yes
Low	Bad	Low	Good	No
Low	Good	Low	Bad	No
High	Good	High	Bad	Yes
Medium	Bad	Low	Bad	No
High	Bad	High	Good	Yes
Low	Bad	High	Bad	No
Medium	Good	Low	Good	No

Showing 9 of 18 training records

CHANDRA'S PROFILE

CGPA

High

Communication

Bad

Aptitude

High

Programming Skill

Job offered? → ?

Bad

THE MATH, PART 1

Entropy: Measuring Impurity

To decide which feature to split on, a decision tree needs a way to measure how "mixed" a group of examples is. That measure is entropy.

$$\text{Entropy}(S) = \Sigma\ -p_i \log_2(p_i)$$

c = number of class labels p_i = proportion of examples in class i

Entropy = 0 → perfectly pure (all examples one class)

Entropy = 1 → maximum impurity (classes evenly split, 2-class case)

Worked example — the full GTS data set (18 students)

Class	Count	p_i	$-p_i \log_2(p_i)$
Yes	8	0.44	0.52
No	10	0.56	0.47

$$\text{Entropy}(S) = -0.44 \cdot \log_2(0.44) - 0.56 \cdot \log_2(0.56) = 0.99$$

THE MATH, PART 2

Information Gain: Picking the Best Split

A tree is built by repeatedly choosing the feature whose split reduces entropy the most — the feature with the highest information gain.

$$\text{Information Gain (S, A)} = \text{Entropy}(S_{\text{aeooe}}\ \text{split}) - \text{Entropy}(S_{\text{aetex}}\ \text{split})$$

Entropy after a split = the weighted sum of entropy in every resulting partition, weighted by each partition's share of the examples.

Gain = 0

The split gave no improvement in purity over the parent node.

Gain > 0

The split increased purity — some information was gained.

Gain = Entropy(before)

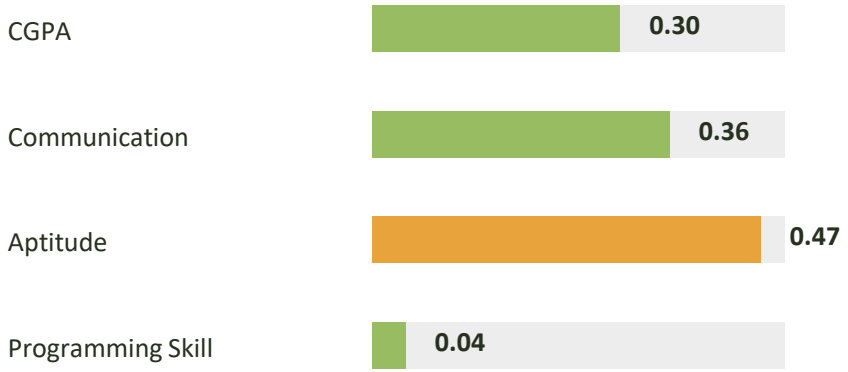
The maximum possible gain — the split produced perfectly pure partitions.

CHOOSING THE ROOT

Level 1: Which Feature Splits Best?

Entropy before any split = 0.99. Each candidate feature is tested by computing entropy after splitting the data on that feature.

Feature	Entropy After Split	Information Gain
CGPA	0.69	0.30
Communication	0.63	0.36
Aptitude	**0.52**	**0.47**
Programming Skill	0.95	0.04



Aptitude wins with the highest information gain (0.47) — it becomes the root node.

Bonus: when Aptitude = Low, entropy is already 0, so that branch needs no further splitting.

CHOOSING THE ROOT

Levels 2 & 3: Growing the Branch

The same entropy / information-gain test is repeated on each remaining branch, using only the examples that reached that branch.

Level 2 — within Aptitude = High

Entropy before split: 0.85

Feature	Entropy After	Info Gain
CGPA	0.33	0.52
Communication	0.30	0.55
Programming Skill	0.80	0.05

Communication gives the highest gain (0.55) → split on Communication.
Communication = Good is already pure (entropy 0).

Level 3 — within Communication = Bad

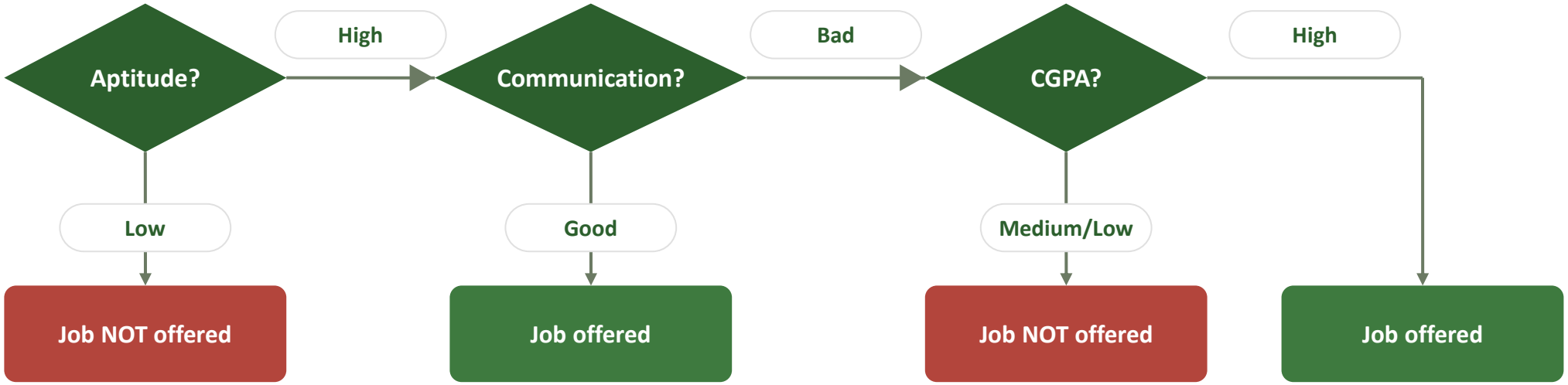
Entropy before split: 0.81

Feature	Entropy After	Info Gain
CGPA	0.00	0.81
Programming Skill	0.50	0.31

CGPA reaches the maximum possible gain (0.81 = entropy before split) → split on CGPA. The branch is now perfectly pure — growth stops.

THE RESULT

The Final Decision Tree for GTS Recruitment



Reading it: the tree needed only three attributes — Aptitude, Communication, and CGPA — to fully classify the training data. Programming Skill never appeared, because it never improved purity enough to be chosen.

APPLYING THE MODEL

Predicting Chandra's Outcome

CGPA = High · Communication = Bad · Aptitude = High · Programming Skill = Bad

STEP 1

Aptitude?

High

Move to Communication branch

STEP 2

Communication?

Bad

Move to CGPA branch

STEP 3

CGPA?

High

Reach leaf node

Prediction: Chandra WILL get the job offer

TRAVERSAL STRATEGIES

Searching a Decision Tree

Once a tree is trained, a new case still needs a path traced through it. Two classic search strategies achieve this.

Exhaustive Search

- Place the item in the first group (class); recursively examine every solution with the item there.
- Place the item in the second group (class); recursively examine every solution with the item there.
- Repeat for every class until a solution is reached.

Guaranteed correct, but slow — costly once the tree has many leaves and attribute values.

Branch and Bound Search

- Starts with a deliberately worst-case “best solution”, so any real result is automatically an improvement.
- Runs down the left-most branch first, then improves from there — sidestepping a full search.
- A fast heuristic can supply a good initial solution, saving substantial time.

Faster than exhaustive search when a good initial heuristic is available.

IMPLEMENTATIONS

Popular Decision Tree Algorithms

Every algorithm builds a tree the same general way — they differ mainly in the impurity measure used to choose each split.

ID3

Iterative Dichotomiser 3

SPLIT MEASURE

Information Gain (Entropy)

C4.5 / C5.0

Successor to ID3

SPLIT MEASURE

Information Gain / Gain Ratio

CART

Classification and Regression Tree

SPLIT MEASURE

Gini Index

CHAID

Chi-square Automatic Interaction Detector

SPLIT MEASURE

Chi-square statistic

MODEL GENERALIZATION

Avoiding Overfitting: Pruning

Left unchecked, a tree keeps splitting until the training data is perfectly classified — overfitting it. Pruning trims the tree so it generalizes to new, unseen data.

1 Pre-Pruning

Stop growing the tree before it reaches perfection — halting at a set number of decision nodes.

+ Advantage: Avoids overfitting and reduces computational cost.

– Trade-off: Risks ignoring important information from a feature that gets skipped, missing real patterns in the data.

2 Post-Pruning

Let the tree grow to its full extent, then trim branches back afterward using a pruning criterion (e.g. error rates at nodes).

+ Advantage: More effective / more accurate — it considers all the minute information available in training data.

– Trade-off: Higher computational cost than pre-pruning.

WRAP-UP

Strengths, Weaknesses & Applications

STRENGTHS

- Produces simple, understandable rules — little math needed for smaller trees
- Works well for most classification problems
- Handles both numerical and categorical variables
- Performs well with small and large training data sets
- Reveals which features are most useful for classification

WEAKNESSES

- Biased toward features with more possible values (levels)
- Gets overfitted or underfitted quite easily
- Prone to errors with many classes and few training examples
- Can be computationally expensive to train
- Large trees are complex to understand

Where it's used: any data set with a finite list of attributes — binary or multi-valued, discrete in nature — and even data with some missing values or classification errors.